

پلان چند-کمپنی

وضعیت: تصویب‌شده، شروع نشده

تاریخ تصمیم: ۱۴۰۵/۰۷/۱۱ (03-10-2026)

برنج در زمان نوشتن: `refactor-items-and-company-wise`

نسخه انگلیسی: `multi-company-plan.md`

اینکه NextBook چطور از یک مشتری با چندین بیزنس پشتیبانی می‌کند — هر بیزنس با نوع، تنظیمات، کتاب‌های مالی و دیتای خودش.

۱. صورت دقیق مسئله

درخواست با این جمله شروع شد: «ما برنج داریم، کمپنی لازم داریم». ولی کد چیز دیگری می‌گوید. برنج در این کدبیس از قبل همان کمپنی است.

`BranchProvisioningService` به هر برنج این‌ها را به‌صورت اختصاصی می‌دهد:

- چارت اکونت کامل و انواع اکونت خودش
- ست اسعار خودش، با `is_base_currency` خودش
- financial period خودش
- کتلاگ اجناس، کتگوری، برند، واحد، مقدار، سائز
- گدام، مشتری نقده، کتگوری مصارف، `journal class`
- کانفیگ HR — شفت، نوع رخصتی، اجزای معاش، جدول مالیه معاش افغانستان
- sequence نمبر بل

این برنج نیست. این یک کمپنی است — حدود ۲۰۰ ردیف seed به‌ازای هر برنج، که با `global scope`‌های `branchSpecific` و `branch` جدا نگه داشته می‌شوند.

در همین حال، جدول `companies` فعلی یک `tenant` نیست. یک ردیف تنظیمات و پروفایل است که به یوزر وصل شده:

`business_type`، `preferences`، `calendar_type`، `locale`، `currency_id`، `costing_method`، تیم بل.

پس پلان این نیست که مفهوم کمپنی را بسازیم. این است که دست از دروغ گفتن درباره کمپنی‌ای که از قبل وجود دارد برداریم، و صفاتی را که یک کمپنی لازم دارد به آن بدهیم.

حفره امنیتی که این وضع باقی گذاشته

جدول `branches` هیچ کالم مالکیت ندارد. `SwitchBranchController` فقط `exists:branches,id` را اعتبارسنجی می‌کند، بدون هیچ محدودیتی. پس یک `super-admin` می‌تواند `branch_id` را روی هر برنچی در دیتابیس بگذارد و بعد همه `global scope`‌ها دیتای آن `tenant` را به او تحویل می‌دهند. بستن این حفره دستاورد اصلی فاز ۲ است.

۲. تصمیم

تغییر نام **companies** → **branches**. یک سطح **tenancy**. یک دیتابیس برای هر مشتری.

دیتابیس = یک مشتری (حتی اگر ۱۰۰ کمپنی داشته باشد)
 کمپنی = یک بیزنس: نوع بیزنس، تنظیمات، کتاب‌های مالی، کتلاگ، جواز،
 سال مالی، اسعار، چارت اکونت — همه از خودش
 + parent_id = کمپنی مادر برای زنجیره (این کالم از قبل روی branches وجود دارد)
 گدام = یک محل نگهداری جنس داخل یک کمپنی

مدل نصب

یک دیتابیس برای هر مشتری، به‌صورت دائمی — حتی بعد از رفتن به SaaS. خود دیتابیس مرز tenant است. نتایجش:

- به هیچ ردیف لنگرگاه tenant ضرورت نیست؛ نود گروه لازم نیست
- **company_id IS NULL** امن است و معنایش «مشترک بین کمپنی‌های این مشتری» است
- هر deploy باید روی همه دیتابیس‌ها migrate شود (بخش ۸ را ببینید)

چرا مدل دوسطحی (کمپنی < برنچ) نه

مدل دوسطحی طراحی و رد شد. آن مدل ایجاب می‌کرد چارت اکونت هر برنچ را در یک چارت سطح-کمپنی dedupe کنیم، که یعنی بازنویسی **transaction_lines.account_id** روی دیتای دوطرفه پست‌شده. این تنها migration در این سیستم است که می‌تواند بی‌صدا تریل بیلانس را جابه‌جا کند. تغییر نام، نام‌گذاری درست را بدون هیچ جابه‌جایی دیتا به دست می‌آورد، و گزینه دوسطحی را از طریق **parent_id** باز نگه می‌دارد.

چه چیزی را از دست می‌دهیم

صادقانه: سه عملیات روزمره، و فقط برای یک زنجیره — یعنی یک مالک با چند دکان هم‌نوع:

۱. کتلاگ به‌ازای هر دکان. یک جنس جدید باید در هر کمپنی جدا ثبت شود. ۲. مشتری با دو بیلانس. کسی که از دو دکان قرضی می‌خرد، دو ledger دارد. سقف اعتبار و **AR aging** به‌ازای هر کمپنی است. ۳. انتقال جنس بین دکان‌ها. **item_transfers** گدام → گدام داخل یک **branch_id** است. بردن جنس بین دکان‌ها یک معامله بین-کمپنی می‌شود، نه یک **transfer**.
- هیچ‌یک از این سه برای بیزنس‌های واقعاً متفاوت مهم نیست (سوپرمارکی و دواخانه چیز به‌کاری با هم شریک ندارند). اگر زنجیره به یک بخش جدی از مشتریان تبدیل شد، **parent_id** به‌علاوه **ancestor scope** هر سه را حل می‌کند، و آن **scope** در یک تابع متمرکز است.

قاعده مرز کمپنی

اگر کتلاگ، مشتریان و کتاب‌های مالی مشترک‌اند → یک کمپنی. اگر مشترک نیستند → کمپنی‌های جدا.

business_type نماینده عملی این قاعده است: هم‌نوع بودن معمولاً یعنی کتلاگ مشترک. توجه کنید که **نمبر جواز معیار نیست** — در افغانستان هر دکان جواز جداگانه خودش را لازم دارد، حتی داخل یک بیزنس. پس جواز روی ردیف کمپنی می‌نشیند و یک زنجیره صرفاً چند کمپنی است که هر کدام جواز خودش را دارد.

۳. سطوح مالکیت دیتا

سطح	مکانیزم	چه چیزی
تعریفی / مشترک	<code>company_id IS NULL</code>	مقادیر، واحدها، سائزها، انواع اکونت، برندها، کانفیگ HR (شفت، نوع رخصتی، اجزای معاش، جدول مالیه)
کمپنی	<code>company_id = X</code>	اجناس و variantها، چارت اکونت، اسعار و نرخها، مشتریان و تأمین‌کنندگان، financial periods، تنظیمات، جواز
محل	<code>warehouse_id</code>	موجودی، tolaها، pieceها

اکونت‌ها و اسعار در سطح کمپنی می‌مانند. این عمده است و یک پیش‌نویس قبلی را نقض می‌کند. هر کمپنی یک مجموعه جداگانه از کتاب‌های مالی است، پس چارت اکونت جداگانه از نظر حسابداری درست است — QuickBooks و Tally هر دو همین‌طور کار می‌کنند — و شریک کردن‌شان ایجاب همان بازنویسی پرخطر `transaction_lines` را می‌کند که این پلان برای فرار از آن ساخته شده.

تکرار تنظیمات با ارث‌بری حل می‌شود، نه با NULL. فاز ۶ را ببینید.

۴. آنچه تغییر نمی‌کند

این‌ها را عمده‌اً ثبت می‌کنیم تا کسی «اصلاح»‌شان نکند:

- `business_type enum` به‌علاوه `config` می‌ماند، هیچ‌وقت جدول نمی‌شود. `config/business_profiles.php` خودش این را گفته: «اضافه کردن یک صنف جدید یعنی اضافه کردن یک کلید اینجا. هیچ‌وقت نباید معنایش یک *migration* باشد.» این موتور پروفایل — که فورم اجناس را بر اساس صنف از قبل شکل می‌دهد — تمایز واقعی NextBook است؛ نه Odoo این کار را می‌کند و نه Tally.
- مالیه معاش افغانستان به‌صورت دیتای seed با تاریخ `effective_from` می‌ماند، تا تغییر نرخ یک ویرایش باشد نه deploy، و دوره‌های گذشته هم بازتولید شوند.
- هیچ فیلد مالیه فروش یا ثبت مالیاتی اضافه نمی‌شود. در بازار افغانستان فعلاً ثبت مالیاتی وجود ندارد. مالیه یک تنظیم کمپنی با پیش‌فرض صفر می‌ماند، تا اگر روزی آمد فقط یک toggle باشد.
- هیچ لایه گروه یا consolidation ساخته نمی‌شود. بدون ثبت مالیاتی مشترک، کسی رپورت ترکیبی قانونی نمی‌خواهد. داشبورد چند-بیزنسی یک رپورت است، نه تغییر schema.

۵. فازها

هر فاز مستقل قابل ارسال است. فقط فاز ۲ بزرگ است، و هیچ فازی دیتای مالی را جابه‌جا نمی‌کند.

فاز ۰ — audit و باگ‌های موجود (بی‌خطر)

۱. کامند `company:audit` که گزارش بدهد: تعداد کمپنی، تعداد برنچ، و اینکه آیا برنچی یوزرهایی از چند کمپنی دارد یا نه. اگر برنچی بین چند کمپنی مشترک بود، متوقف شوید — آن مورد تصمیم دستی لازم دارد. ۲. رفع سه باگ که همین حالا وجود دارند:

- مدل `Company SoftDeletes` ندارد و جدول `companies` کالم `deleted_at` ندارد، ولی `routes/web.php:123` مسیر

`companies.restore ... withTrashed` () را باز گذاشته.

- `Administration/CompanyController::index` متود `Company::search` () را صدا می‌زند؛ مدل `HasSearch` trait را ندارد.
- مدل‌های `Account` و `Currency` فیلد `tenant_id` را در `fillable$` و `casts` () دارند و `CurrencyResource` هم برش می‌گرداند، ولی هیچ migrationی این کالم را نمی‌سازد. مرده است — حذف شود. ۳. آپشن `--tenant=` در `SeedDemoData` به `--company=` تغییر کند. بعد از این پلان، «tenant» یعنی دیتابیس، و آن آپشن در واقع `company id` می‌گیرد.

فاز ۱ — افزودن صفات کمپنی به `branches` (کم‌خطر، صرفاً افزودنی)

این کالم‌ها به `branches` اضافه می‌شوند، همه nullable، هیچ چیزی حذف نمی‌شود:

```
business_type, calendar_type, locale, currency_id, costing_method,
preferences (json), logo, abbreviation, name_fa, name_pa,
email, website, address, phone, country, city,
invoice_description, invoice_theme
جواز: license_number, license_issued_at, license_expires_at
سال مالی: fiscal_year_start_month, fiscal_year_start_day,
timezone, is_active, deleted_by
```

backfill هر برنج از ردیف `companies` قدیم، که از طریق یوزرهای همان برنج پیدا می‌شود. اضافه کردن

`unique(['license_number', 'deleted_at'])`.

نکات جواز: جواز به کمپنی (= دکان) تعلق دارد، و اسناد چاپی باید جواز کمپنی صادرکننده را نشان بدهند. `license_expires_at` می‌تواند یک یادآور تجدید را از طریق preference موجود `document_expiry_alert` فعال کند.

فاز ۲ — تغییر نام (خطر متوسط، مکانیکی)

ترتیب مهم است — نام `companies` باید اول آزاد شود.

۱. `companies` → `legacy_company_settings` (یک نسخه نگه داشته شود، بعد حذف) ۲. `branch_id` ۳. `branches` → `company_id` روی تمام ۹۶~ جدول، همراه با `index` ها و `FK` ها ۴. ادغام `users.branch_id` و `users.company_id` در یک `unique key` ۵. `users.company_id` های `['users: ['company_id', 'email', 'deleted_at']` و `['company_id', 'username', 'deleted_at']` ۶. محدود کردن سویچر کمپنی به کمپنی‌هایی که یوزر عضوشان است. حفره امنیتی اینجا بسته می‌شود.

نقشه تغییر نام کد:

از	به
<code>BranchSpecific + BelongsToBranch</code> (دو trait، یک کار)	یک <code>BelongsToCompany</code>
<code>BranchContext</code>	<code>PostingContext</code>
<code>SetActiveBranch</code>	<code>SetActiveCompany</code>
<code>active_branch_id</code>	<code>active_company_id</code>
<code>BranchScopedUnique</code>	<code>CompanyScopedUnique</code>
<code>SwitchBranchController</code>	<code>SwitchCompanyController</code>
<code>()BasePolicy::sameBranch</code>	<code>()sameCompany</code>
<code>BranchProvisioningService</code>	<code>CompanyProvisioningService</code>

اندازه کار، اندازه‌گیری شده در 03-10-2026:

- 1327 مورد `branch_id` در 382 فایل PHP
- 31 مورد در 12 فایل JS/Vue — فرانت‌اند تقریباً دست نمی‌خورد
- 103 فایل تست به برنج اشاره دارند — این‌ها تور محافظ شما هستند
- 40 فایل ترجمه (en / fa / ps)

دو ساده‌سازی از این کار بیرون می‌آید: آن یک `trait` باقی‌مانده جای دو `trait` را می‌گیرد که ناهماهنگ به‌کار رفته بودند، و `PostingContext` آن `memoization` به‌ازای-برنج برای ~۴۰ تا `slug` اکونت را از دست می‌دهد، چون جست‌وجوی `slug` الآن در سطح کمپنی بدون ابهام است.

یک رفتار که همراه تغییر نام باید اصلاح شود: `BranchSpecific` وقتی هیچ برنچی `bind` نشده باشد از `scope` بیرون می‌زند، پس کامندهای `console` و `queue worker`ها ردیف‌های همه `tenant`ها را می‌بینند. جایش یک `Tenancy::runFor($companyId, fn` (`=>` صریح بگذارید که `job`ها مجبور باشند صدایش بزنند، و در غیر آن با خطای واضح شکست بخورد. همه `dispatch()`ها باید بازبینی شوند که `company id` را `serialize` می‌کنند.

فاز ۳ — یوزر در چند کمپنی (کم‌خطر)

```
company_user: company_id, user_id, is_default, joined_at
PRIMARY KEY (company_id, user_id)
```

`users.company_id` به‌عنوان «آخرین کمپنی فعال» می‌ماند. سویچر کمپنی در `topbar`، که تنها وقتی نشان داده می‌شود که `pivot` بیش از یک ردیف داشته باشد. سویچر باید عضویت را اعتبارسنجی کند، کمپنی را در `session` بگذارد، و کش‌ها را پاک کند — آن لیست دستی ~۲۵ تا `forget()->cache()` در سویچر فعلی با یک `tagged flush` عوض شود.

به‌علاوه یک داشبورد «همهٔ بیزنس‌ها»: `whereIn('company_id', $memberCompanyIds)`. این یک لایهٔ `consolidation` نیست، فقط یک رپورت است.

فاز ۴ — ویزارد ساخت کمپنی (کم‌خطر)

چهار قدم: هویت و جواز → نوع بیزنس → اسعار، تقویم، سال مالی → گدام اول.

قدم نوع بیزنس یک پیش‌نمایش زنده نشان می‌دهد که آن صنف چه چیزهایی را روشن می‌کند، که مستقیماً از `config/business_profiles.php` خوانده می‌شود. اگر نمبر جواز تکراری بود هشدار بدهد: احتمالاً منظورشان اضافه کردن گدام به یک کمپنی موجود بوده.

در `submit`، یک `transaction`: کمپنی → ردیف `CompanyProvisioningService->provision` → `company_user()` که از پروفایل `seed` می‌گیرد.

`Provisioning` به سه بخش تقسیم می‌شود:

سرویس	چند بار	چه می‌سازد
<code>SystemProvisioningService</code>	یک بار برای هر دیتابیس	ستهای تعریفی مشترک با NULL (فاز ۵)
<code>CompanyProvisioningService</code>	هر کمپنی	چارت اکونت، انواع اکونت، اسعار، <code>financial period</code> ، مشتری نقده، کتگوری مصارف، <code>journal classes</code>
provisioning گدام	هر گدام	ردیف گدام اصلی

فاز ۵ — NULL-share کردن ستهای تعریفی (کم‌خطر)

`company_id` را nullable کنید و به یک ردیف مشترک در هر دیتابیس `dedupe` کنید برای: `quantities`، `unit_measures`، `sizes`.

account_types, brands, و کانفیک shifts, leave_types, salary_components, tax_bracket_sets (HR و bracketها).

این‌ها امن‌اند چون foreign key هایشان کم و غیرمالی است. scope این می‌شود:

```
$q->where('company_id', $activeCompanyId)->orWhereNull('company_id');
```

بعد از این، nullable بودن company_id خودش یک بیان در سطح schema است که آن جدول دیتای قابل اشتراک دارد. جدول‌های تراکنشی NOT NULL می‌مانند.

صریحاً در این فاز نیستند: اکونت‌ها و اسعار (بخش ۳).

فاز ۶ — resolver تنظیمات (کم‌خطر)

چهار لایه:

پیش‌فرض‌ها	config/preferences.php
business_type به‌ازای	business profile →
سیاست سطح زنجیره	→ کمپنی مادر (parent_id)
این بیزنس	→ کمپنی
فقط موارد شخصی	→ یوزر

قدم‌ها:

۱. انتقال ۴۱۵، (User.php:54-469) User::DEFAULT_PREFERENCES (سطر) به config/preferences.php، تقسیم‌شده به دو لیست صریح company_scoped و user_scoped. سطح کمپنی: prefix بل، مالیه، قواعد پست، تأییدیه‌ها، فیلدهای جنس، امنیت، بکاپ. سطح یوزر: ظاهر، تیم، اندازه فونت، تعداد رکورد در صفحه، صداها، سایدبار، ۲. onboarding. یک resolver جدید App\Support\Settings که در هر request یک بار محاسبه شود. BusinessProfile یک لایه داخل آن می‌شود، نه یک سیستم موازی. ۳. بازنویسی PreferencesController که فعلاً فقط users.preferences را می‌خواند و می‌نویسد. نوشتن یک کلید سطح کمپنی به اجازه company.settings.update ضرورت دارد. ۴. بازنویسی CoreShared تا user_preferences همان merge نهایی باشد.

لایه کمپنی مادر همان چیزی است که تکرار تنظیمات را حل می‌کند: مشتری‌ای که یک کمپنی با سه برنج داشت، به سه کمپنی تبدیل می‌شود که هر کدام یک کپی از preferences دارند. یک بار روی مادر تنظیم کنید و هر سه ارث می‌برند. این اولین جایی است که parent_id واقعاً ارزش خودش را نشان می‌دهد.

فاز ۷ — ماژول‌ها و رول‌ها (خطر متوسط)

۱. اضافه کردن یک گروه modules به هر ['hr' => true, 'pos' => false, 'manufacturing' => true], business profile — [...] — و تغذیه آن هم به سایدبار و هم به یک EnsureModuleEnabled middleware، تا یک دواخانه نتواند با URL به manufacturing/ برسد. نوع بیزنس فعلاً فقط فیلدهای جنس را کنترل می‌کند. ۲. فعال کردن Spatie teams با team_foreign_key company_id = فعلاً در config/permission.php مقدار 'teams' => false است، پس رولی که در یک کمپنی ساخته شود در کمپنی دیگر هم قابل تخصیص است. permission جهان می‌ماند (رشته‌هایی هستند که در کد تعریف شده‌اند)؛ رول‌ها به سطح کمپنی می‌آیند. primary key pivot کالم company_id می‌گیرند، پس این یک بازسازی جدول است: بساز، کپی کن، عوض کن. migration استوک، team key را unsignedBigInteger تعریف می‌کند — برای ULID خودتان بنویسید. ۳. سه سطح: platform-admin به‌عنوان یک flag روی یوزر برای دسترسی پشتیبانی بین-کمپنی (فعلاً super-admin در SetActiveBranch و سویچر با مقایسه رشته چک می‌شود)؛ company-admin به‌ازای هر کمپنی؛ و رول‌های عملیاتی پایین‌تر. در زمان ساخت هر کمپنی یک ست رول پیش‌فرض seed شود.

۶. سؤالات باز

۱. آیا مشتری‌ای با چند دکان هم‌نوع وجود دارد؟ اگر بله، ارث‌بری تنظیمات از `parent_id` (فاز ۶) از همان اول لازم است، نه در فاز ۶.
۲. نمبرگذاری بل در یک زنجیره. پیش‌فرض، sequence به‌ازای هر کمپنی است. اگر زنجیره‌ای یک sequence با prefix هر دکان خواهد (HasSequentialNumber)، (`INV-KBL-`, `INV-HRT-`) به یک کلید ترکیبی ضرورت دارد. ۳. نرخ تبادله.
- `currency_rate_updates.branch_id` به `company_id` تبدیل می‌شود، پس هر کمپنی نرخ‌های خودش را دارد. برای یک زنجیره یعنی وارد کردن نرخ امروز دالر به‌ازای هر دکان. کاندید مناسبی برای ارث‌بری از مادر.

۷. این تصمیم از کجا آمد

پیش از تصمیم، پنج سیستم مقایسه شد. همه روی یک شکل توافق دارند:

سیستم	سطح کتاب‌ها	سطح محل
QuickBooks	company file (subscription جدا)	Location / Class تگ
Xero	organisation	tracking category
Odoo	<code>res.company</code> (یک DB، درخت <code>parent_id</code>)	warehouse / analytic account
SAP	(company code (BUKRS	plant (WERKS) → storage location
NetSuite	subsidiary	location
Tally	فایل دیتای company	cost centre / godown

دو نتیجه طراحی را هدایت کرد:

- هیچ سیستمی master data را زیر سطح کتاب‌ها نگه نمی‌دارد. NextBook فعلاً این کار را می‌کند — و تغییر نام دقیقاً همین را اصلاح می‌کند، با یکی کردن سطح کتاب‌ها و سطح دیتا.
- هیچ سیستمی یک محل را کمپنی نمی‌نامد. از همین‌جا قاعدهٔ مرز در بخش ۲ آمد.

عمداً قرض گرفته شد: مکانیزم `company_id IS NULL` از Odoo و سویچر چند-کمپنی هم‌زمانش؛ و از Tally سرعت ورود دیتا با کیبورد و شروع بدون کانفیگ، که دلیل واقعی وفاداری کاربران در این بازار است و مدل درست برای ماژول `PosSession` (POS) فعلاً یک مدل بدون controller و بدون صفحه است). عمداً قرض گرفته نشد: مدل دیتای فایل-به-ازای-کمپنی Tally، و عمق کانفیگ Odoo.

۸. کارهای عملیاتی که این تصمیم ایجاب می‌کند

یک دیتابیس برای هر مشتری رایگان نیست. پیش از مشتری دهم:

۱. یک کامند `migrate` که روی همهٔ دیتابیس‌های مشتریان اجرا شود و گزارش بدهد کدام ناکام ماند. بدون آن، نسخه‌های schema بی‌صدا و اگر می‌شوند. ۲. یک registry مرکزی از مشتریان و connection‌هایشان، یا یک فایل کانفیگ به‌ازای هر نصب. ۳. یک runbook برای بکاپ و restore به‌ازای هر دیتابیس.